

Master Thesis

Design and Evaluation of Efficient Semantic Company Retrieval at Scale

Marmee Pandya
(matriculation number 1963521)

July 24, 2026

Submitted to
Chair of Enterprise Data Analysis
Dr. Ralph Peeters
University of Mannheim

Abstract

Your thesis must contain an abstract. A good reference for thesis writing is [Zobel \(2014\)](#); we highly recommend that you study this or a similar book during your studies. He writes the following about the abstract:

An abstract is typically a single paragraph of about 50 to 200 words. The function of an abstract is to allow readers to judge whether or not the paper is of relevance to them. It should therefore be a concise summary of the paper's aims, scope, and conclusions. There is no space for unnecessary text; an abstract should be kept to as few words as possible while remaining clear and informative. Irrelevancies, such as minor details or a description of the structure of the paper, are inappropriate, as are acronyms, abbreviations, and mathematics. Sentences such as "We review relevant literature" should be omitted.

The more specific an abstract is, the more interesting it is likely to be. Instead of writing "space requirements can be significantly reduced", write "space requirements can be reduced by 60%". Instead of writing "we have a new inversion algorithm", write "we have a new inversion algorithm, based on moveto-front lists".

Many scientists browse research papers outside their area of expertise. You should not assume that all likely readers will be specialists in the topic of their paper-abstracts should be self-contained and written for as broad a readership as possible. Only in rare circumstances should an abstract cite another paper (for example, when one paper consists entirely of analysis of results in another), in which case the reference should be given in full, not as a citation to the bibliography. ([Zobel 2004](#))

Contents

Abstract	ii
1 Project Overview	1
2 ISTARI.AI and its Data	2
2.1 About ISTARI.AI	2
2.2 The Global Organisation Index (GOI)	2
2.3 Data Pipeline and Methodology	2
2.3.1 Schema and Key Fields	3
2.3.2 Industry Classification: NACE Codes	4
2.3.3 Organisation Size Classification	4
2.3.4 Geographic Coverage	5
2.3.5 Evaluation Dataset	5
3 Literature	6
3.1 Sparse and Dense Retrieval	6
3.2 Approximate Nearest Neighbour Search	7
3.3 Late Interaction and Retrieval-Specific Pretraining	7
3.4 Hybrid Retrieval, Reranking, and Learning to Rank	7
3.5 Evaluation Without Human Relevance Judgements	8
4 Experiments	9
4.1 Experimental Setup	9
4.1.1 Dataset	9
4.1.2 Evaluation Protocol	10
4.1.3 Computational Environment	10
4.2 Sparse Retrieval Baseline: BM25	11
4.2.1 Implementation	11
4.2.2 Qualitative Analysis	11
4.3 Dense Retrieval Baselines	11
4.3.1 all-MiniLM-L6-v2 (MiniLM)	12
4.3.2 BGE-large-en-v1.5 (BGE)	12
4.3.3 OpenAI text-embedding-3-small (OpenAI)	12
4.4 Results	13
4.4.1 Retrieval Quality	13
4.5 Runtime Analysis	14
4.6 Evaluation Protocol Change for Follow-Up Experiments	16
4.7 Extended Baseline Comparison: Additional Embedding Models	16

Contents

4.8	BGE Instruction-Prefix Correction	18
4.9	Query Expansion: HyDE Pipeline	19
4.10	Hybrid Retrieval: Reciprocal Rank Fusion	20
4.11	LLM-as-Judge Relevance Validation	20
4.12	Production API Evaluation and MiniLM Rerank	21
4.13	ANN Index Scaling to 2 Million Vectors	22
4.14	Recall@1000 Error Analysis	22
4.15	Corpus Quality: Platform Boilerplate in the AI-Generated Summary . . .	23
4.16	Oracle Union Recall Ceiling	24
4.17	SimLM: A Purpose-Built Retrieval Pre-training Baseline	25
4.18	ColBERT: Late-Interaction Retrieval from Scratch	26
4.19	Hybrid Full-Pool Reranking	27
4.20	Reranker Ablation: Isolating the Bottleneck	28
4.21	Learned Fusion Ranker	29
	Bibliography	31
	Ehrenwörtliche Erklärung	33

1 Project Overview

Thesis Goal: Build and evaluate an efficient embedding-based semantic retrieval system for large-scale company data that outperforms the current production search system.

Core Question: Can embedding-based semantic retrieval combined with approximate nearest neighbor search efficiently generate accurate top-k candidate company sets from large-scale datasets while maintaining low query latency?

Current Todo List: - Literature Review

2 ISTARI.AI and its Data

2.1 About ISTARI.AI

This thesis is conducted in collaboration with ISTARI.AI¹, a data intelligence company that specialises in providing high-quality firmographic data at scale. Istari’s core product is a curated, organisation-level dataset that covers organisations worldwide, enabling use cases such as market analysis, company search, and lead generation. The central research challenge addressed in this thesis is improving the quality of semantic company retrieval, arises directly from Istari’s operational needs.

2.2 The Global Organisation Index (GOI)

The primary dataset used in this thesis is Istari’s **Global Organisation Index** (GOI), a curated collection of approximately 20 million active organisations spanning 232 countries and territories. The GOI serves as the canonical source of firmographic data within Istari’s data infrastructure, covering organisation identity, location, industry classification, size, etc.

2.3 Data Pipeline and Methodology

The GOI is constructed through a multi-stage validation pipeline that deliberately prioritises data quality over volume. The pipeline proceeds as follows:

1. **Registry Collection.** National registers are systematically queried worldwide, including commercial registers, company registers, and association registers. These primary sources are enriched with additional open data sources, yielding an initial pool of approximately 400 million organisations.
2. **Domain Attribution.** Each organisation is assessed for attribution to a clearly identifiable web domain. Approximately 10% of the initial pool, around 40 million organisations can be matched to a domain. The remainder are excluded as they are either no longer active, were never truly operational (e.g., pure holding structures), or lack a web presence.
3. **Activity Verification.** The 40 million domain-attributed organisations are further filtered by verifying that their domains are still actively operated. This final

¹https://docs.istari.ai/goi/goi_start_page/

step reduces the dataset to approximately 20 million organisations that are demonstrably active.

This pipeline ensures that every record in the GOI is both web domain-verified and confirmed active, a key differentiator from traditional firmographic databases, which typically retain large numbers of inactive or dormant records.

2.3.1 Schema and Key Fields

Each record in the GOI contains a set of core identity fields, geographic fields, and classification fields. The fields most relevant to this thesis are summarised in Table 2.1.

Table 2.1: Key fields of the GOI schema used in this thesis.

Field	Type	Description
name	STRING	Organisation name
domain	STRING	Organisation website domain (unique identifier)
summary	STRING	AI-generated textual summary of the organisation
keywords	LIST	Keywords
employee_class	STRING	Employee count bracket
country	STRING	Country
country_code	STRING	ISO Country Code
state / state_code	STRING	State or province
region / region_code	STRING	Region
district / district_code	STRING	District
municipality / municipality_code	STRING	Municipality
address	STRING	Full address
latitude	STRING	Latitude coordinate
longitude	STRING	Longitude coordinate
nace_code	STRING	NACE industry classification code
organization_type	STRING	Company, Startup, Academic, Public, or Other
organization_size	STRING	Micro, Small, Medium-sized, or Large enterprise
country	STRING	Country of registration

Of particular importance for this thesis is the **summary** field. This field contains an AI-generated natural language description of each organisation, synthesised from its web presence and available structured data. It is the most semantically rich and consistently populated field in the dataset, with effectively zero missing values across the corpus used for experiments, and therefore serves as the primary input for all retrieval models evaluated in this thesis.

2.3.2 Industry Classification: NACE Codes

Industries in the GOI are classified using the **NACE** taxonomy² (*Nomenclature statistique des activités économiques dans la Communauté européenne*), the European Union’s standard statistical classification of economic activities. The GOI covers 22 NACE sectors. Table 2.2 lists the ten most represented sectors in the dataset.

Table 2.2: Top 10 industries in the GOI by organisation count.

NACE	Industry	Count	Share (%)
N	Professional, scientific & technical activities	3,093,879	18.03
G	Wholesale & retail trade	2,240,191	13.05
C	Manufacturing	2,006,435	11.69
R	Human health & social work	1,440,340	8.39
K	Telecom, IT & computing	1,269,131	7.39
F	Construction	1,173,226	6.84
I	Accommodation & food service	1,126,594	6.56
S	Arts, sports & recreation	1,101,592	6.42
Q	Education	584,430	3.41
J	Publishing, broadcasting & content	554,985	3.23

2.3.3 Organisation Size Classification

Organisation size in the GOI is derived from employee and revenue signals and bucketed into four categories following the European Union’s definitions of small and medium-sized enterprises (SME), as shown in Table 2.3.

Table 2.3: Organisation size distribution in the GOI.

Size	Employee Range	Count	Share (%)
Micro	0–9	7,341,838	42.78
Small	10–49	7,494,409	43.67
Medium-sized	50–249	1,501,655	8.75
Large enterprise	250+	824,126	4.80

The distribution is heavily skewed towards smaller organisations: over 86% of the GOI consists of micro and small enterprises. This has implications for retrieval, as queries targeting large enterprises represent a smaller and more distinctive subset of the corpus.

²<https://ec.europa.eu/eurostat/web/nace>

2.3.4 Geographic Coverage

The GOI covers 232 countries and territories. Table 2.4 lists the top five countries by volume.

Table 2.4: Top 5 countries by organisation count in the GOI.

Country	Organisations
United States	3,626,794
Germany	1,828,181
United Kingdom	1,013,026
Netherlands	778,416
Italy	625,754

It is worth noting that approximately 2.7 million organisations in the GOI do not have a standardised administrative region. These records are intentionally retained as they remain valuable for non-geographic analyses such as industry or domain-level retrieval, the primary focus of this thesis.

2.3.5 Evaluation Dataset

The full 20-million-company GOI corpus is not directly used for retrieval in this thesis. Instead, Istari provided an evaluation dataset consisting of 101 real-world search queries along with each query’s top-1 000 production search results, yielding 101 000 (query, company) pairs in total. Where the same domain appears linked to multiple addresses, the GOI deduplicates by domain and retains the record with the highest employee count, treating it as the organisation’s headquarters; after this deduplication, the evaluation corpus covers **98 716 unique companies**. All retrieval experiments in Chapter 4 are conducted over this corpus. The exact pseudo-relevance threshold used to define “relevant” companies for evaluation (and how it changes across experiments) is described in Sections 4.1.2 and 4.6.

Testing retrieval quality at the full 20-million-company scale was not pursued in the current phase of this thesis, since doing so would require a substantially larger data export from Istari than the query-level top-1 000 results used here; this is discussed as an open question in the concluding chapter.

3 Literature

This chapter reviews the retrieval paradigms, indexing techniques, and evaluation practices that this thesis builds on and compares against. Section 3.1 covers sparse and dense retrieval, the two foundational paradigms for the baselines in Chapter 4. Section 3.2 covers approximate nearest neighbour search, the technique that makes dense retrieval practical at scale. Section 3.3 covers late-interaction retrieval and retrieval-specific pre-training, the two alternative paradigms evaluated in Sections 4.17–4.18. Section 3.4 covers hybrid retrieval, cross-encoder reranking, and learning to rank, the techniques underlying the hybrid pipelines in Sections 4.19–4.21. Section 3.5 covers evaluation practice in information retrieval when human relevance judgements are unavailable, directly motivating the pseudo-relevance protocol used throughout this thesis (Section 4.1.2).

3.1 Sparse and Dense Retrieval

Text retrieval systems fall into two broad families. **Sparse** retrieval represents queries and documents as high-dimensional vectors of term weights and scores matches by term overlap. The dominant sparse method is BM25 (Robertson and Zaragoza 2009), a probabilistic scoring function that weighs each query term by its frequency in a document and its rarity across the whole collection. BM25 is fast, requires no training, and remains a strong baseline decades after its introduction, but it cannot match a query and a document that describe the same concept using different words: this is the vocabulary mismatch problem, directly observed for this thesis’s task in Section 4.2.2.

Dense retrieval instead represents queries and documents as fixed-size vectors learned by a neural encoder, and scores matches by vector similarity rather than literal term overlap. Dense Passage Retrieval (DPR) (Karpukhin et al. 2020) established the now-standard bi-encoder architecture for open-domain retrieval: a query encoder and a document (or, in this thesis, company) encoder, trained so that relevant pairs are close in vector space, encoding each side independently so that documents can be embedded once, offline, and reused across every future query. Sentence-BERT (Reimers and Gurevych 2019) showed that the same bi-encoder idea, applied to general-purpose sentence embeddings rather than a retrieval-specific training objective, produces embeddings useful for semantic similarity and retrieval tasks broadly, and underlies the general-purpose sentence-transformer models (including MiniLM, Section 4.3.1) used throughout this thesis. More recent embedding models, such as the BGE family (Xiao et al. 2024; Chen et al. 2024), combine this bi-encoder architecture with large-scale contrastive pretraining and hard-negative mining, and represent the current state of the art for general-purpose dense retrieval, forming the basis of several baselines evaluated in Sections 4.3.2 and 4.7.

3.2 Approximate Nearest Neighbour Search

Dense retrieval requires finding the nearest vectors to a query vector among the full document collection. Exact search costs time linear in the number of documents, which becomes impractical at the scale of millions of documents relevant to this thesis’s target deployment (Section 2). Approximate nearest neighbour (ANN) search trades a small amount of accuracy for a large reduction in query time by searching only a carefully chosen subset of the collection instead of comparing the query against every document. FAISS (Johnson et al. 2019) provides GPU-accelerated implementations of both exact and approximate search, including inverted-file indexes (IVF), which cluster the embedding space and search only the clusters nearest the query, trading a small amount of recall for a large reduction in query time. Hierarchical Navigable Small World graphs (HNSW) (Malkov and Yashunin 2020) take a different approach: they build a multi-layer proximity graph over the document vectors and greedily traverse it at query time, again trading exactness for speed. Both IVF and HNSW are evaluated directly in this thesis’s ANN scaling experiment (Section 4.13), which tests whether the accuracy–speed trade-off these methods make remains favourable at simulated production scale.

3.3 Late Interaction and Retrieval-Specific Pretraining

A bi-encoder compresses an entire document into one fixed-size vector, which can discard fine-grained information relevant to a specific query. ColBERT (Khattab and Zaharia 2020) addresses this with *late interaction*: query and document are each represented as a set of per-token vectors, and relevance is scored by summing, for every query token, the maximum similarity to any document token (MaxSim), so no single vector has to summarise the whole document in advance. This adds compute and storage cost relative to a single-vector bi-encoder, since retrieval now requires comparing token sets rather than single vectors. ColBERTv2 (Santhanam et al. 2022) introduces residual compression of the per-token vectors and the PLAID engine for efficient late-interaction search at scale, substantially reducing this cost; this thesis instead implements MaxSim scoring directly (Section 4.18), trading the PLAID engine’s efficiency for an exact, dependency-light implementation suited to the scale of the evaluation corpus used here.

A separate line of work improves dense retrieval not through architecture but through pretraining. SimLM (Wang et al. 2022) pretrains a bi-encoder with a representation-bottleneck objective before any retrieval-specific fine-tuning, explicitly designed to produce embeddings well suited to downstream passage retrieval. This thesis evaluates whether that retrieval-specific pretraining transfers to a domain (company profiles) quite different from the passage-retrieval benchmarks it was designed for (Section 4.17).

3.4 Hybrid Retrieval, Reranking, and Learning to Rank

Sparse and dense retrieval, and different dense models, make different errors: a hybrid system that combines multiple retrieval signals can recover from a single method’s in-

dividual weaknesses. Reciprocal Rank Fusion (RRF) (Cormack et al. 2009) is a simple, training-free way to combine several ranked lists: each document’s fused score is the sum of $1/(k + \text{rank})$ across the lists it appears in, requiring only that a document be ranked, not that its raw scores be comparable across methods. RRF underlies the hybrid baseline in Section 4.10.

A complementary technique is *reranking*: a first-stage retriever returns a candidate set cheaply, and a more expensive model reorders it. Cross-encoder rerankers, such as the `bge-reranker-v2-m3` (BAAI 2024) and `mxbai-rerank-large-v1` (Mixedbread AI 2024) models used in Sections 4.19 and 4.20, feed the query and each candidate into the same transformer jointly, rather than encoding them independently as a bi-encoder does. This lets the model attend directly between query and candidate tokens, typically producing more accurate relevance scores than a bi-encoder, at the cost of requiring a full forward pass per candidate rather than a single fast vector comparison, so cross-encoders are used to rerank a small candidate set rather than to search a full collection.

More generally, combining multiple retrieval signals into a single relevance score can be framed as *learning to rank* (Liu 2009): instead of a fixed, hand-designed combination rule such as RRF, a model is trained directly on labelled query-document pairs to learn how to weight and combine whatever features are available (similarity scores, ranks, or other signals) into a final ranking. This is the approach taken by the learned fusion ranker in Section 4.21, implemented with the logistic regression and gradient-boosted tree models provided by scikit-learn (Pedregosa et al. 2011).

3.5 Evaluation Without Human Relevance Judgements

Standard information retrieval evaluation compares each system’s ranking against a set of human relevance judgements for a fixed query set, historically constructed through pooling: the top results from several systems are merged and judged by human assessors, as established by the methodology of the Text REtrieval Conference (TREC) (Voorhees 1999). No such human-annotated relevance judgements exist for the company search task addressed in this thesis. In their absence, this thesis instead adopts a *pseudo-relevance* protocol (Section 4.1.2): a company is treated as relevant if it appears in the existing production system’s own results for that query. This is a common practice when human judgements are unavailable, and the BEIR benchmark (Thakur et al. 2021) similarly demonstrates that retrieval methods can be compared consistently across many tasks and domains without task-specific human annotation, provided the resulting scores are interpreted as agreement with a reference system rather than as absolute measures of relevance, a limitation this thesis states explicitly wherever pseudo-relevance labels are used.

4 Experiments

This chapter describes the experimental setup, baseline retrieval systems, evaluation methodology, and results obtained for the company search task over Istari’s Global Organisation Index (GOI). Section 4.1 introduces the dataset and evaluation protocol. Sections 4.2–4.3 describe each retrieval baseline in turn. Section 4.4 presents and analyses the results across all methods, and Section 4.5 provides a runtime comparison. Section 4.6 documents a change in evaluation protocol adopted for all subsequent experiments. Sections 4.7–4.16 extend the baseline comparison, explore query expansion and hybrid fusion, validate the pseudo-relevance labels against a large language model (LLM) judge, compare against Istari’s production application programming interfaces (APIs), evaluate approximate nearest neighbour search at scale, analyse the sources of retrieval error, and document a corpus-level data-quality limitation affecting the AI-generated company summaries used throughout this thesis. Sections 4.17–4.18 evaluate two further retrieval paradigms (a retrieval-pretrained encoder and late-interaction retrieval). Sections 4.19–4.21 develop a full-pool reranking pipeline, isolate the reranker as a bottleneck, and close part of the remaining gap with a learned fusion ranker trained on this task’s own data, the best-performing method in this thesis.

4.1 Experimental Setup

4.1.1 Dataset

The evaluation corpus is derived from Istari’s production search logs. A set of 101 representative queries was provided by Istari, covering a wide range of company search intents, including industry-specific queries (e.g. “*pharmaceutical companies in the US*”), geographic queries (e.g. “*tech startups in Berlin*”), and functional queries (e.g. “*B2B SaaS companies*”). For each query, the production system returned its top-1 000 ranked results, yielding 101 000 query–company pairs in total. After de-duplication across queries, the corpus consists of **98 716 unique companies**, each represented by a domain name, a company name, and an AI-generated natural-language summary of the company’s business activity.

The summary field was selected as the primary retrieval field after an exploratory data analysis of all available GOI attributes. It was found to be fully populated across the corpus (0% missing values) and to contain the richest semantic content for matching against free-text queries, compared to structured fields such as industry codes or founding year.

4.1.2 Evaluation Protocol

Pseudo-relevance labels. No human-annotated relevance judgements are available for the Istari company search task. Following standard practice in information retrieval research when ground-truth labels are absent (Voorhees 1999), this thesis adopts a *pseudo-relevance* assumption: a company is considered **relevant** for a given query if and only if it appears in the **top-100 results** of Istari’s production search system for that query. This yields a relevance set of at most 100 companies per query, representing $\sim 0.1\%$ of the total corpus.

This assumption has an important implication for the evaluation: the metrics measure the degree to which each retrieval system agrees with the production system, not whether it recovers objectively relevant companies. Consequently, a retrieval system that is strictly better than the production system may appear to underperform because it surfaces companies the production system missed. This limitation is acknowledged throughout the analysis.

Metrics. Three standard information retrieval metrics are reported at cut-offs $k \in \{10, 50, 100, 1000\}$:

- **Precision@ k** : the fraction of the top- k retrieved companies that are relevant. This measures user-facing quality: what a user sees on the first page of results.
- **Recall@ k** : the fraction of all relevant companies that appear in the top- k results. This measures coverage, important for downstream re-ranking pipelines that operate on a retrieved candidate set. **Recall is the primary metric in this thesis**, following the priority set by Istari for this task: whether the system finds the relevant companies matters more than the exact order they are ranked in. Section 4.6 explains how Precision@ k and Recall@ k relate to one another and should be read together.
- **NDCG@ k** (Normalised Discounted Cumulative Gain): a ranking-aware metric that rewards systems for placing relevant companies higher in the list, using a logarithmic discount. NDCG@ k is reported as a secondary, supporting metric: it reflects the experience of a user who interacts primarily with the top results, which is a reasonable concern in practice even though it is not the priority metric for this thesis.

All metrics are averaged across all 101 queries. Precision and Recall are always reported at the same cut-off values so that the precision–recall trade-off at each depth is visible.

4.1.3 Computational Environment

All experiments are conducted on the University of Mannheim’s high-performance computing (HPC) cluster. Dense model encoding is performed on an NVIDIA A100 graphics

processing unit (GPU) with 80 GB HBM2 memory. BM25 retrieval runs on the central processing unit (CPU) instead, since it does not involve any neural network computation. Query latency measurements are performed on the same hardware and reported as the mean over 20 benchmark queries to obtain stable estimates. Latency is decomposed into three components (local query encoding, FAISS index search, and external network time) so that self-hosted and API-based methods are compared on equal terms.

4.2 Sparse Retrieval Baseline: BM25

4.2.1 Implementation

The BM25 baseline is implemented using the `rank_bm25` Python library (Robertson and Zaragoza 2009), which provides the Okapi BM25 variant with default hyperparameters $k_1 = 1.5$ and $b = 0.75$. All 98 716 company summaries are tokenised using the punkt tokeniser from the Natural Language Toolkit (NLTK) (Bird et al. 2009) with lower-casing applied, and the resulting token lists are used to construct the BM25 inverted index. At query time, each query is tokenised identically and scored against the index. The top-1 000 companies by BM25 score are retrieved for each query and stored for evaluation. Index construction takes 28.7 seconds and requires no GPU.

4.2.2 Qualitative Analysis

Manual inspection of BM25 results reveals a consistent failure mode arising from the *vocabulary mismatch problem*: BM25 retrieves companies that contain query keywords in their description, regardless of whether those companies operate in the queried domain.

A representative example is the query “*software companies*”. The top BM25 results are predominantly investment banks and venture capital firms whose summaries frequently mention the word *software* in the context of describing their investment portfolio (e.g. “*we invest in software, fintech, and SaaS businesses*”). These companies are not software companies: they fund them. BM25 cannot distinguish between a company *operating* in a domain and a company *discussing* that domain. This limitation provides a clear and concrete motivation for semantic retrieval methods that operate on the meaning of text rather than on token overlap.

4.3 Dense Retrieval Baselines

Three dense retrieval baselines are evaluated, covering a range of model sizes, computational requirements, and deployment constraints. All three follow the same two-stage pipeline introduced by Karpukhin et al. (2020): (i) encode all company summaries offline into a dense vector index using FAISS (Johnson et al. 2019), and (ii) at query time, encode the query into a vector and retrieve the top-1 000 nearest neighbours from the index.

4.3.1 all-MiniLM-L6-v2 (MiniLM)

`all-MiniLM-L6-v2` is a lightweight sentence-transformer model (Reimers and Gurevych 2019) with 22 million parameters, producing 384-dimensional embeddings. It is distilled from a larger BERT model and fine-tuned on sentence similarity tasks. All 98 716 company summaries are encoded in a single batch pass, taking 0.6 minutes on the A100 GPU. A `faiss.IndexFlatL2` index (exact Euclidean search) is constructed over the resulting embeddings in 0.2 seconds. MiniLM does not require embedding normalisation.

MiniLM is included as the smallest and most computationally efficient dense baseline, providing a lower bound on dense retrieval quality and establishing whether a minimal dense model is sufficient to outperform BM25.

4.3.2 BGE-large-en-v1.5 (BGE)

`BAAI/bge-large-en-v1.5` is a 335-million parameter BERT-large model developed by the Beijing Academy of Artificial Intelligence (Xiao et al. 2024), producing 1 024-dimensional embeddings. It is specifically optimised for dense retrieval through hard negative mining during fine-tuning, a training strategy in which the model is exposed to query–document pairs that are superficially similar but semantically distinct, forcing it to learn fine-grained semantic distinctions.

Encoding all 98 716 company summaries takes 7.0 minutes on the A100 GPU at a batch size of 256. BGE requires L2 normalisation of all embeddings (`normalize_embeddings=True`), as it is trained under the cosine similarity objective. Accordingly, a `faiss.IndexFlatIP` index (inner product, equivalent to cosine similarity for unit-normalised vectors) is used rather than `IndexFlatL2`.

At query time, each query is encoded independently and normalised before being passed to the FAISS index. The query encoding step takes 16.6 ms on the A100 GPU, and the subsequent FAISS search takes 27.8 ms, for a total query latency of 44.4 ms.

4.3.3 OpenAI text-embedding-3-small (OpenAI)

`text-embedding-3-small` is a commercial embedding model provided by OpenAI via their REST API. It produces 1 536-dimensional embeddings. Unlike the open-source models above, the architecture and training procedure are proprietary. Encoding all 98 716 company summaries is performed via batched API calls (batch size 500) and takes 9.7 minutes, of which the majority is network latency rather than model computation. The resulting embeddings are unit-normalised by the API, and a `faiss.IndexFlatIP` index is constructed locally. The total cost of encoding the full corpus is approximately \$0.15 at the published pricing of \$0.02 per million tokens.

This model is included as the commercial upper-bound baseline, following a suggestion from the industry supervisor that a high-quality API-based embedding model be evaluated alongside open-source alternatives.

Latency note. Query latency for the OpenAI baseline comprises two components: a network round-trip to the OpenAI API servers (211.7 ms, measured over 20 queries)

and a local FAISS search (42.9 ms). The network component is not a property of the model itself; it reflects internet latency between the University of Mannheim cluster and OpenAI’s infrastructure, and should not be interpreted as a measure of model speed. For a fair comparison, the local FAISS search time (42.9 ms) is comparable to BGE’s search time (27.8 ms), confirming that both methods perform equivalently at the index search stage. The substantive difference is that OpenAI requires an external API call for query encoding, whereas BGE encodes locally on GPU in 16.6 ms.

4.4 Results

4.4.1 Retrieval Quality

Recall is the primary metric reported in this thesis, following the priority set by Istari for this task: coverage of relevant companies matters more than precise ranking of the first page of results. Table 4.1 presents Recall@ k for all four methods; NDCG and Precision are discussed alongside it in the paragraphs below, where individual values are relevant, rather than tabulated in full.

Table 4.1: Recall@ k of all four baseline systems averaged over 101 queries. Bold indicates the best value in each column.

Method	Recall@10	Recall@50	Recall@100	Recall@1000
BM25	0.016	0.068	0.128	0.682
MiniLM	0.019	0.069	0.122	0.653
BGE	0.022	0.087	0.159	0.796
OpenAI	0.022	0.080	0.149	0.803

BM25 vs dense retrieval. All three dense methods outperform BM25 at low cut-offs. At $k = 10$, BGE and OpenAI achieve an NDCG of 0.227, representing a **37% improvement** over BM25’s 0.166. Precision@10 improves from 0.156 (BM25) to 0.216 (BGE), a 38% relative gain. This confirms the hypothesis that semantic retrieval better captures the intent of natural-language company search queries, particularly for cases where relevant companies use vocabulary different from the query terms.

At large cut-offs ($k = 1000$), the gap narrows. BM25 achieves a Recall@1000 of 0.682, compared to BGE’s 0.796 and OpenAI’s 0.803. This pattern is consistent with findings in the BEIR benchmark (Thakur et al. 2021): dense models are stronger at ranking precision, while keyword-based methods maintain competitive coverage at large retrieval depths due to their broad lexical matching.

BGE vs OpenAI. BGE and OpenAI are **effectively equivalent** across all metrics and cut-offs. The largest observed difference is 0.013 NDCG points (at $k = 50$), which is within the range of query-level variance and should not be interpreted as a meaningful

4 Experiments

quality advantage. This finding has an important practical implication: a free, self-hosted model (BGE) achieves the same retrieval quality as a commercial API (OpenAI) on this task.

MiniLM. MiniLM consistently underperforms BGE and OpenAI at all cut-offs, and is only marginally better than BM25 at large k (Recall@1000: 0.653 vs 0.682 for BM25). It is also slower than BGE in production (query latency 80.9 ms vs 44.4 ms), as discussed in Section 4.5. MiniLM is therefore dominated by BGE on both quality and speed, and there is no operating point at which it represents the optimal choice.

Precision and the low-prevalence setting. Absolute precision values are low across all methods (Precision@10 ranging from 0.156 to 0.224). This is expected: relevant companies represent at most 100 out of 98 716 corpus entries, approximately 0.1% of the corpus. In such a low-prevalence setting, even a perfect retrieval system would show low absolute precision at large k . What matters is the relative improvement between methods: BGE and OpenAI achieve 38–43% higher Precision@10 than BM25, which is a meaningful gain.

The NDCG dip at $k = 100$. An artifact of the evaluation design is visible in Table 4.1: NDCG values decrease from $k = 10$ to $k = 100$ before rising sharply at $k = 1000$ for all methods. This occurs because the pseudo-relevance labels are defined as the production top-100. At exactly $k = 100$, NDCG measures whether all 100 relevant companies are ranked perfectly within the top-100 retrieved results, a maximally strict condition. At $k = 1000$, there is substantially more room to accommodate the relevant set, and the metric rises accordingly. This artifact would not appear with human-annotated relevance labels that include varying degrees of relevance beyond a hard top-100 cut-off.

4.5 Runtime Analysis

Table 4.2 presents the decomposed query latency for all four methods. Latency is broken into three components to enable a fair comparison between self-hosted models and the API-based OpenAI baseline.

BGE is the fastest method overall. BGE achieves a total query latency of 44.4 ms, faster than BM25 (104.1 ms), MiniLM (80.9 ms), and OpenAI (254.6 ms). This result is initially counterintuitive: BGE is a 335-million parameter neural network, whereas BM25 is a simple keyword scoring function. The explanation lies in hardware utilisation. BGE encodes the query vector on the A100 GPU (16.6 ms), a highly parallelised operation that is extremely efficient for dense linear algebra. BM25, by contrast, computes token scores on CPU over an inverted index (104.0 ms), which is sequential and memory-bound. This finding has a direct implication for production deployment: a GPU-equipped system running BGE delivers both better retrieval quality and lower query latency than a CPU-based BM25 system.

4 Experiments

Table 4.2: Query latency decomposed into local encoding, FAISS search, and external network time, averaged over 20 benchmark queries on an NVIDIA A100 GPU. Network time reflects internet latency to OpenAI servers and is not a property of the model.

Method	Encode (ms)	Search (ms)	Network (ms)	Total (ms)	Self-hostable
BM25	0.1	104.0	0.0	104.1	Yes
MiniLM	68.9	12.0	0.0	80.9	Yes
BGE	16.6	27.8	0.0	44.4	Yes
OpenAI	<i>server</i>	42.9	211.7	254.6	No

MiniLM encodes slowly despite being a small model. MiniLM’s 68.9 ms encoding time is slower than BGE’s 16.6 ms, despite having 15 times fewer parameters. This is because MiniLM encodes on CPU in these experiments, while BGE encodes on GPU. Hardware assignment dominates over model size for single-query encoding latency.

OpenAI’s latency is dominated by network time. Of OpenAI’s total 254.6 ms, only 42.9 ms is FAISS index search; comparable to BGE’s 27.8 ms search time. The remaining 211.7 ms is the round-trip time of the Hypertext Transfer Protocol (HTTP) request to OpenAI’s servers. This latency is not a property of the embedding model; it reflects internet connectivity between the university cluster and OpenAI’s infrastructure, and would vary across deployment environments. For a production system at Istari, API-based encoding would introduce an irreducible network dependency and per-query cost, whereas BGE could be self-hosted with predictable latency.

Summary. Table 4.3 summarises the key trade-offs across all four methods.

Table 4.3: Summary comparison of all baseline retrieval methods.

Method	NDCG@10	Prec@10	Recall@1k	Latency	Cost	GPU req.
BM25	0.166	0.156	0.682	104 ms	Free	No
MiniLM	0.192	0.186	0.653	81 ms	Free	No
BGE	0.227	0.216	0.796	44 ms	Free	Yes (enc.)
OpenAI	0.227	0.224	0.803	255 ms	\$0.02/M tok.	No

BGE emerges as the recommended baseline for subsequent experiments: it matches OpenAI’s retrieval quality, is the fastest method in practice, is free, and can be fully self-hosted without API dependency. The results motivate the next experimental phase, in which a pre-filtering stage and hybrid retrieval system are built on top of the BGE baseline to further improve precision at low cut-offs and overall system efficiency.

4.6 Evaluation Protocol Change for Follow-Up Experiments

All experiments described in the remainder of this chapter (Sections 4.7–4.16) use a **relaxed pseudo-relevance definition**: a company counts as relevant if it appears anywhere in production’s **top-1 000** results for a query, rather than the top-100 threshold used in Section 4.1.2 and Table 4.1. The relaxed threshold was adopted so that reranking, fusion, and oracle-ceiling experiments would have enough relevant companies per query to produce meaningful comparisons over large candidate pools. As a direct consequence, the absolute NDCG/Precision/Recall values reported from here on are on a different (generally much higher) numerical scale than Table 4.1, and **are not directly comparable to it**. All comparisons in the following sections should be read only relative to one another, within the same section.

Reading Precision and Recall together. Under this relaxed protocol, every one of the 101 queries has exactly 1 000 relevant companies. This has a direct consequence for how Precision@ k and Recall@ k relate to one another at any cut-off $k \leq 1\,000$: since both metrics are computed from the same count (how many of the top- k retrieved companies are relevant), just divided by a different denominator,

$$\text{Recall@}k = \text{Precision@}k \times \frac{k}{1\,000},$$

so the two metrics are exactly proportional to one another at every cut-off, and neither is redundant on its own: Precision@ k answers “of what the system returned, how much was correct”, while Recall@ k answers “of everything that is correct, how much did the system return”, and the two only coincide once $k = 1\,000$, where the denominators become equal. A practical consequence is that Recall@ k for small k can look misleadingly low in isolation. For example, a system that returns 10 companies of which 8 are genuinely relevant has an excellent Precision@10 of 0.80, but the corresponding Recall@10 is only $0.80 \times 10/1\,000 = 0.008$, not because the 10 companies returned are poor, but because 10 items can never cover more than 1% of a 1 000-company relevant set. For this reason, the tables in this chapter report Precision@ k at the shallow cut-offs ($k \in \{10, 50, 100, 300, 500\}$), where it is the more directly interpretable number, and report both Precision@1 000 and Recall@1 000 together, since $k = 1\,000$ is the one cut-off at which Recall stops being a rescaled copy of Precision and becomes a genuinely independent measure: total coverage of the full relevant set. Recall@1 000 is treated as the headline metric throughout this thesis for exactly this reason.

4.7 Extended Baseline Comparison: Additional Embedding Models

Five further dense embedding models were evaluated on the same 98 716-company corpus, all-fields text representation, and 101-query evaluation set described in Section 4.1.1:

4 Experiments

- **BGE-M3** (BAAI/bge-m3): a multi-lingual, multi-functionality embedding model, 1 024-dimensional embeddings.
- **GTE-large** (thenlper/gte-large): a general-purpose text embedding model, 1 024 dimensions.
- **E5-Mistral-7B-Instruct** (intfloat/e5-mistral-7b-instruct): an instruction-tuned 7-billion-parameter Mistral-based embedding model, 4 096 dimensions.
- **Linq-Embed-Mistral** (Linq-AI-Research/Linq-Embed-Mistral): a 7B-parameter Mistral-based embedding model fine-tuned for retrieval, 4 096 dimensions.
- **SFR-Embedding-Mistral** (Salesforce/SFR-Embedding-Mistral): Salesforce’s 7B-parameter Mistral-based retrieval embedding model, 4 096 dimensions.

Table 4.4: Extended baseline Recall@ k under the relaxed (top-1 000) pseudo-relevance protocol (Section 4.6). Bold indicates the best value in each column.

Method	Precision@ k				Recall@1000
	10	50	100	1000	
BGE-M3	0.927	0.916	0.905	0.704	0.704
GTE-large	0.946	0.936	0.925	0.768	0.768
E5-Mistral-7B	0.629	0.572	0.523	0.274	0.274
Linq-Mistral	0.934	0.921	0.915	0.749	0.749
SFR-Mistral	0.933	0.922	0.915	0.746	0.746

Findings. GTE-large is the strongest of the five additional models (Recall@1000 = 0.768, NDCG@10 = 0.946), ahead of Linq-Mistral (0.749 / 0.936), SFR-Mistral (0.746 / 0.938), and BGE-M3 (0.704 / 0.935). These four sit close together and clearly ahead of E5-Mistral-7B-Instruct, which substantially underperforms (Recall@1000 = 0.274, NDCG@10 = 0.631) despite sharing the same base architecture and parameter count as Linq-Mistral and SFR-Mistral. This indicates that fine-tuning data and strategy, not model size, are the dominant factor for this retrieval task. Notably, the two much smaller 1 024-dimensional models (BGE-M3, GTE-large) are competitive with or better than the 7B-parameter models, at a fraction of the encoding cost.

Encoding cost. The three 7B-parameter models required substantially more compute to encode the corpus: E5-Mistral took 2 011 seconds (33.5 min), Linq-Mistral 2 039 seconds (34.0 min), and SFR-Mistral 2 032 seconds (33.9 min), all on an A100 GPU, and all requiring checkpoint/resume logic to fit within the cluster’s 30-minute job wall-time limit. This is roughly five times the encoding time of the original 335M-parameter BGE-large baseline (7.0 minutes, Section 4.3.2), for retrieval quality that is not better and, in E5-Mistral’s case, substantially worse.

Incomplete and abandoned models. Two further large embedding models were attempted but did not produce final results. **BGE-Multilingual-Gemma2** (BAAI/bge-multilingual-gemma2, 9B parameters) encoding is still in progress at the time of writing: model loading alone takes 15–27 minutes on this cluster’s shared storage (highly variable due to filesystem I/O contention), leaving little of the 30-minute job window for actual encoding, so the job must be resubmitted repeatedly to make incremental progress via checkpointing. **NV-Embed-v2** (nvidia/NV-Embed-v2) was abandoned after a persistent `AttributeError` caused by an incompatibility between the model’s custom code and the installed `transformers` library version, which could not be resolved without a library downgrade that risked breaking the other baselines’ environment.

4.8 BGE Instruction-Prefix Correction

The original BGE-large baseline (Section 4.3.2) encoded queries without an instruction prefix. BGE-large’s model card specifies that, for retrieval tasks, each *query* should be prefixed with the instruction “*Represent this sentence for searching relevant passages:* ” before encoding; the passage (company) side requires no prefix, so the cached company embeddings could be reused without re-encoding the corpus. This fix was retested on both the all-fields and summary-only variants of the BGE baseline.

Table 4.5: Effect of the query instruction prefix on BGE-large, measured under the relaxed top-1 000 protocol (Section 4.6).

Variant	NDCG@10	Recall@1000	Δ NDCG@10	Δ Recall@1000
All-fields, no prefix	0.942	0.726	–	–
All-fields, with prefix	0.937	0.727	–0.005	+0.001
Summary-only, no prefix	0.937	0.700	–	–
Summary-only, with prefix	0.924	0.707	–0.014	+0.007

Finding. Contrary to expectation, applying the officially documented query prefix produced only a marginal and mixed effect on this dataset: Recall@1000 improved negligibly (+0.001 for all-fields, +0.007 for summary-only), while NDCG@10 slightly *decreased* in both variants. Both effects are small enough to plausibly be within query-level noise. This rules out the missing prefix as the explanation for a broader pattern seen throughout this chapter: under the relaxed protocol, BGE (0.726 all-fields, 0.700 to 0.707 summary-only) and the related BGE-M3 model (0.704, Section 4.7) both reach a lower Recall@1000 than the much smaller, general-purpose MiniLM model (0.741, Table 4.6), despite BGE’s larger parameter count and its retrieval-specific fine-tuning.

Why a specialised, larger model can still be a worse fit. A plausible explanation lies in what each model was optimised for during training, rather than in how it is queried at inference time. BGE-large and BGE-M3 are fine-tuned through hard negative mining

4 Experiments

(Section 4.3.2) on benchmark data dominated by short, single-topic passages retrieved for open-domain question answering, an objective that sharpens the model’s embedding space around the specific pattern of a question matched against a topically focused prose passage. MiniLM, by contrast, is trained as a general-purpose sentence-similarity model on a broad mixture of paraphrase and natural language inference data (Reimers and Gurevych 2019), an objective that rewards recognising that two pieces of text mean similar things across a wide range of domains and writing styles, rather than one narrow retrieval pattern. The company records retrieved in this thesis are not passages: each one concatenates heterogeneous, only loosely prose-like fields (organisation name, country, state, city, organisation type and size, industry code) with a short free-text summary. This structure is closer to the varied, mixed-format text that MiniLM’s training was designed to generalise across than to the passage-retrieval benchmarks that shaped BGE’s embedding space. Fine-tuning a model tightly around one task and one kind of text is well documented to improve performance on that specific benchmark while sometimes reducing robustness on text the model was not tuned for (Thakur et al. 2021), and the results in this chapter are consistent with that trade-off: BGE’s retrieval-specific specialisation does not transfer as cleanly to this heterogeneous company-record format as MiniLM’s broader, less specialised training does.

4.9 Query Expansion: HyDE Pipeline

HyDE (Hypothetical Document Embeddings) expands each query by asking a local LLM (llama3.1:8b via Ollama) to generate $N = 10$ stochastic (temperature = 0.8) hypothetical company descriptions that would ideally answer the query. The embeddings of these 10 descriptions are averaged into a single query vector, which is then searched against the MiniLM company index. A second stage reranks the top-50 HyDE results with a cross-encoder (BAAI/bge-reranker-v2-m3).

Table 4.6: HyDE pipeline results (relaxed top-1 000 protocol) compared against the plain MiniLM baseline re-evaluated under the same protocol.

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no expansion)	0.975	0.741	16.9
Stage 1: HyDE + MiniLM	0.829	0.596	9 246.9
Stage 2: HyDE + MiniLM + Reranker	0.954	0.596	9 488.3

Finding. Query expansion via HyDE *hurts* retrieval quality relative to directly encoding the original query: Stage 1 NDCG@10 drops from 0.975 to 0.829, and Recall@1000 drops from 0.741 to 0.596. Adding a cross-encoder reranker (Stage 2) recovers most of the NDCG@10 loss ($0.829 \rightarrow 0.954$) by re-ordering the top-50 candidates, but cannot recover the recall loss, since reranking only reorders an already -fixed candidate set. Latency is dominated overwhelmingly by the LLM call itself (9.2 seconds), three orders

of magnitude slower than directly encoding the query with MiniLM (16.9 ms). For this dataset, HyDE is not a worthwhile query-expansion strategy: it is far slower and retrieves a worse candidate set than simply embedding the query directly.

4.10 Hybrid Retrieval: Reciprocal Rank Fusion

As an alternative to query expansion, BM25 and MiniLM rankings are fused directly using Reciprocal Rank Fusion (RRF), $\text{RRF}(d) = \sum_{\text{method}} \frac{1}{k + \text{rank}(d)}$ with $k = 60$ (Cormack et al. 2009). A second stage reranks the top-50 fused results with the same cross-encoder used for HyDE.

Table 4.7: Hybrid RRF results (relaxed top-1 000 protocol).

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no fusion)	0.975	0.741	16.9
Stage 1: RRF (BM25 + MiniLM)	0.955	0.724	104.9
Stage 2: RRF + Reranker	0.975	0.724	352.0

Finding. Unlike HyDE, RRF fusion of BM25 and MiniLM adds negligible latency (the fusion step itself takes 2.2 ms) and, after reranking, matches the plain MiniLM baseline’s NDCG@10 (0.975) while Recall@1000 remains slightly below MiniLM alone (0.724 vs 0.741): BM25’s keyword matches do not add companies beyond what MiniLM already retrieves for this query set, so fusion mostly re-confirms rather than extends MiniLM’s candidate pool. RRF is a much cheaper and more effective way to incorporate a second retrieval signal than HyDE-style query expansion.

4.11 LLM-as-Judge Relevance Validation

Because the pseudo-relevance labels (Section 4.1.2) assume production’s own rankings are correct, an independent check was run using an LLM to directly judge relevance. For 20 held-out queries (from a pool of 101, split 81/20 train/held-out), the top-20 production results per query were judged individually on a 0/1/2 relevance scale by an LLM given the query and the company summary, yielding 400 graded judgements in total.

Finding. At $k = 10$, pseudo-relevance and LLM-judged NDCG agree closely (0.964 vs 0.963), supporting the use of production’s own top results as a reasonable relevance proxy at shallow depths. The two diverge sharply as k grows: LLM-judged NDCG stays essentially flat at 0.982 out to $k = 1000$, while pseudo-relevance NDCG falls to 0.771. This is consistent with the pseudo-relevance labels being a hard top-100 cutoff (Section 4.1.2): once retrieval looks past that cutoff, pseudo-relevance necessarily penalises

Table 4.8: NDCG under pseudo-relevance labels vs. LLM-judged relevance labels, on the 20 held-out queries.

k	Pseudo-relevance NDCG	LLM-judged NDCG	Δ
10	0.964	0.963	-0.001
50	0.947	0.982	+0.035
100	0.939	0.982	+0.042
500	0.868	0.982	+0.113
1000	0.771	0.982	+0.210

the system even for companies an LLM judge considers genuinely relevant. This supports the interpretation, already flagged in Section 4.1.2, that pseudo-relevance NDCG at large k understates true system quality rather than reflecting a real quality gap.

4.12 Production API Evaluation and MiniLM Rerank

Two rounds of evaluation compared the local pipeline against Istari’s live production search API, using a separate ground truth (the production system’s own graded semantic-search results at a similarity threshold of 0.75) rather than the pseudo-relevance labels used elsewhere in this chapter.

v1: keyword API. The first round compared Istari’s v1 (BM25-based) production API against local MiniLM, both queried against the same 101 queries. Istari v1 BM25 NDCG@10 was 0.101 and Recall@1000 was 0.109 (network latency 2 828 ms/query), against local MiniLM’s NDCG@10 of 0.935 and Recall@1000 of 0.766 (local GPU latency 26.6 ms/query), confirming that the production keyword API is a much weaker retrieval signal than even the simplest local dense baseline.

v2: semantic API. A second round added Istari’s newer v2 semantic search API (live, paginated over the full 20-million-company GOI, quota-aware). Istari v2 NDCG@10 was 0.985 and Recall@1000 (at the achievable depth of 816 results) was 0.650, clearly ahead of local MiniLM (NDCG@10 0.935, Recall@816 0.709) at shallow cutoffs, though local MiniLM’s Recall catches up and slightly exceeds v2’s at $k = 816$. Network latency for v2 (1 966 ms/query for a full fetch) remains far higher than local MiniLM (26.4 ms/query).

Reranking v2’s own candidates with MiniLM. A follow-up experiment took v2’s own retrieved candidate pool and re-ordered it locally using MiniLM, to test whether local reranking could improve on v2’s native ranking without changing which companies are retrieved. This made results *worse*: NDCG@10 fell from 0.985 (v2 native ranking) to 0.855 (MiniLM-reranked). Istari’s v2 semantic ranking is not simply a MiniLM-style dense embedding search under the hood: it outperforms MiniLM’s own ranking of the same candidates.

4.13 ANN Index Scaling to 2 Million Vectors

To test whether the exact search used throughout this chapter (`IndexFlatL2/IndexFlatIP`) remains practical at production scale, the 98 716 real MiniLM company embeddings were padded with 1 901 284 synthetic filler vectors (real embeddings perturbed with Gaussian noise at 15% of the average vector norm), giving a 2-million-vector corpus. Exact search (`FlatL2`) was compared against approximate indexes: IVF (`nlist=4096`, with `nprobe` $\in \{1, 8, 32, 128, 4096\}$) and HNSW ($M = 32$, with `efSearch` $\in \{16, 64, 256\}$).

Table 4.9: Exact vs. approximate search at 2M-vector scale (relaxed top-1 000 protocol). Only representative configurations shown.

Index	NDCG@10	Recall@1000	ms/query	Speedup	Index size
Exact (FlatL2)	0.975	0.741	175.5	1.0×	3 072 MB
IVF (nprobe=8)	0.974	0.743	26.4	6.6×	3 094 MB
HNSW (efSearch=256)	0.975	0.727	26.9	6.5×	3 616 MB

Finding. Both IVF (`nprobe=8`) and HNSW (`efSearch=256`) recover essentially all of exact search’s NDCG@10, and IVF slightly exceeds exact search’s Recall@1000 (0.743 vs 0.741, within noise), while running 6.5–6.6× faster at 2-million-vector scale, at the cost of modest additional index memory (0.7% and 17.7% larger, respectively). This confirms that approximate search is a safe and substantial efficiency gain for scaling the retrieval system toward the full 20-million-company GOI, without a meaningful quality trade-off at this scale.

4.14 Recall@1000 Error Analysis

To understand why exact MiniLM search caps at Recall@1000 = 0.741 (Section 4.13), every production-relevant company that MiniLM failed to retrieve within the top-1 000 was extracted and searched for at much greater depth ($k = 10\,000$), yielding 26 185 missed (query, company) pairs across all 101 queries.

Table 4.10: Classification of the 26 185 Recall@1000 misses by how far beyond rank 1 000 the true company was actually ranked.

Bucket	Count	Share (%)
Near miss (rank 1 000–2 000)	16 157	61.7
Mid miss (rank 2 000–5 000)	8 061	30.8
Far miss (rank 5 000–10 000)	1 375	5.3
Not found (beyond rank 10 000)	592	2.3

Finding. The large majority of misses (61.7%) are near misses, just outside the top-1 000 cutoff, suggesting most of the recall gap is a matter of ranking granularity rather than a hard semantic failure. Only 2.3% of misses are not found at all within $k = 10\,000$, indicating genuine embedding failures are rare. Separately, 3.8% of missed companies (996 of 26 185) were flagged as boilerplate content, and one extreme case of description duplication was found: the placeholder text “Bluehost.com” appears as the company description for 709 distinct domains, meaning these domains are indistinguishable to any text-embedding model and can never be individually retrieved correctly. This is a concrete data-quality artifact rather than a retrieval-model limitation, and sets a hard ceiling on achievable recall for any embedding-based method operating on the `summary` field alone.

4.15 Corpus Quality: Platform Boilerplate in the AI-Generated Summary

The duplicated “Bluehost.com” description found in Section 4.14 is one instance of a broader, previously unquantified data-quality issue affecting the `summary` field that every retrieval method in this thesis depends on. For a meaningful share of companies in the corpus, the AI-generated summary describes the website hosting provider or website-building platform that the company’s domain happens to run on, rather than the company’s own business. This happens because many small businesses have a thin, template-driven web presence, so the process that generates each company’s summary from its web content sometimes picks up generic platform language (for example, marketing copy for a web hosting provider or an e-commerce storefront builder) instead of any information specific to the company itself. A company using such a platform is then, in effect, indistinguishable in its summary text from every other company using the same platform, which is precisely the mechanism behind the large-scale description duplication already observed in Section 4.14.

How much of the corpus is affected. This contamination is not spread evenly across query topics. An audit of the pseudo-relevance labels for all 101 queries found that, for the query “*web hosting companies*”, 85.4% of the 1 000 ground-truth relevant companies have a boilerplate rather than a company-specific summary; for “*cloud computing providers*” the figure is 73.1%, and for “*e-commerce platforms*” it is 53.1%. By contrast, most other query topics in the evaluation set (among them “*software companies*”, “*healthcare providers*”, and “*construction companies*”) show no measurable contamination at all. This pattern is intuitive once the underlying mechanism is understood: boilerplate summaries describe hosting, domain, and e-commerce infrastructure, so they are only likely to be retrieved, and only likely to be judged relevant by the production system’s own ranking, for queries whose intent is itself adjacent to that infrastructure. A query about software companies is unlikely to surface a domain registrar’s boilerplate text, but a query about web hosting companies is exactly the case in which that boilerplate text looks, on the surface, like a plausible match.

An attempt to detect it automatically. A classifier was built to flag boilerplate summaries automatically, using embedding similarity between each company’s summary and a small, fixed reference set of around 300 known boilerplate templates, an approach chosen so that classifying any single company costs one embedding comparison against a fixed-size reference set, regardless of whether the underlying corpus is the 98 716-company evaluation set used in this thesis or the full multi-million-company GOI. This classifier was validated against 300 companies independently judged by an LLM as having a trustworthy or untrustworthy summary. At its best-performing similarity threshold, the classifier reaches an F1 score of 0.59, with a precision of 0.52 and a recall of 0.69 for detecting untrustworthy summaries.

Why this limitation was not corrected in the reported results. A precision of 0.52 means that of every two summaries the classifier flags as boilerplate, roughly one is, in fact, a genuine, company-specific summary. Removing or down-weighting every flagged company from the retrieval pipeline would therefore discard a large number of companies that are not actually contaminated, trading one source of error for another rather than removing it. For this reason, no correction for boilerplate contamination was applied to any of the results reported in this thesis, and all recall and precision figures should be read with this limitation in mind, particularly for the small set of query topics identified above as heavily affected. This is reported here as an explicitly known and quantified, but unaddressed, limitation of the underlying corpus rather than as a solved problem: a natural direction for future work is to use the boilerplate-similarity score not as a hard filter but as an additional input feature to a trained model, such as the learned fusion ranker introduced in Section 4.21, which is able to learn how much to trust an imperfect signal rather than requiring it to be reliable enough for a fixed decision rule.

4.16 Oracle Union Recall Ceiling

Before investing in a trained reranker or a fine-tuning pipeline, it is useful to know the best-case recall achievable by simply combining the candidate pools already produced by every method run so far — at zero additional encoding cost, since every channel’s top-1 000 results per query are already cached on disk. This experiment computes, for every combination of the six available channels (MiniLM, BM25, BGE-all-fields, BGE-summary, OpenAI-large, Nomic), the recall of the *union* of their top-1 000 candidate sets against the top-1 000 ground truth.

Findings. The best single channel (MiniLM alone) reaches $\text{Recall@1000} = 0.741$. The full union of all six channels reaches 0.936 (average combined pool size 1 949 candidates/query), a gain of +0.196 over MiniLM alone. The best-performing *pair* is MiniLM + OpenAI-large, at 0.856 (average pool size 1 300). A pool-size sweep on this best pair shows the ceiling is highly sensitive to how many candidates are taken per channel: at 100 candidates/channel the union only reaches 0.150 recall, rising to 0.581 at 500, and 0.856 at the full 1 000.

Interpretation. Because the six-channel union clears 0.936, well above any individual channel, the different retrieval methods are largely making *complementary*, not correlated, errors: a reranker operating over a modestly sized union pool (rather than a single channel) is likely sufficient to approach this ceiling, and a full fine-tuning pipeline is not obviously necessary to close the remaining gap. The residual 0.064 recall lost even under the full six-channel union is consistent with the hard data-quality ceiling identified in Section 4.14 (duplicated/boilerplate company descriptions that no embedding-based channel can disambiguate).

4.17 SimLM: A Purpose-Built Retrieval Pre-training Baseline

All dense baselines evaluated so far are general-purpose sentence-embedding models. **SimLM** (Wang et al. 2022) takes a different approach: it is a BERT-base (110 M parameter) encoder pre-trained specifically for passage retrieval using a representation-bottleneck objective, before any task-specific fine-tuning. It is included to test whether a model purpose-built for retrieval, but far smaller than the 7B-parameter models in Section 4.7, can match or exceed them.

Implementation. The public checkpoint `intfloat/simlm-base-msmarco-finetuned` is used, following its documented convention rather than the all-fields concatenation used elsewhere in this thesis: the company name is encoded as a *title*, and the remaining fields (country, state, city, type, size, industry, summary, keywords) as the paired *body* text, using CLS-token pooling (not mean pooling) with L2 normalisation. Queries are encoded without any prefix, following the same model card.

Table 4.11: SimLM Recall@1000 (relaxed top-1 000 protocol) alongside the strongest baselines evaluated so far, for reference.

Method	Precision@ <i>k</i>				Recall@1000
	10	50	100	1000	
BM25 (Section 4.2)	0.838	0.748	0.723	0.635	0.635
SimLM	0.920	0.881	0.861	0.609	0.609
MiniLM (Section 4.3.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.7)	0.946	0.936	0.925	0.768	0.768

Finding. SimLM underperforms every dense baseline evaluated in this thesis, and its Recall@1000 (0.609) falls even below the lexical BM25 baseline (0.635), despite SimLM being purpose-pretrained for passage retrieval. (NDCG@10 tells a less clear-cut story here: SimLM’s 0.923 sits above BM25’s 0.854, since NDCG rewards ranking quality among whichever companies are retrieved, not coverage of the full relevant set.) A likely explanation is a domain mismatch between SimLM’s pre-training convention and

this task: SimLM was pre-trained and fine-tuned on MS MARCO, a corpus of short, single-topic passages, using a title/body sentence-pair format that assumes the title is a concise topical label. Company names are not topical labels in the same sense: “Sweet Software” or “Springcoast Capital Partners” convey little about what the paired body text describes, so the sentence-pair convention that helps SimLM on MS MARCO-style data may provide a weaker signal here than simply concatenating all fields into one string, as the other baselines do. This result is a useful negative finding: retrieval-specific pre-training does not automatically transfer across domains, and general-purpose sentence embeddings (MiniLM, GTE-large) remain the stronger choice for this task.

4.18 ColBERT: Late-Interaction Retrieval from Scratch

Every method evaluated so far represents each company and each query as a single dense vector. **ColBERT** (Khattab and Zaharia 2020; Santhanam et al. 2022) instead represents both as *sets of per-token* embeddings and scores a (query, company) pair using *MaxSim*: for every query token, the maximum similarity to any company token is taken, and these per-token maxima are summed. This is architecturally richer than single-vector dense retrieval, since no single fixed-size vector has to compress the entire meaning of a company description, and it is included to test whether that added expressiveness translates into higher recall on this task.

Implementation. The public checkpoint `colbert-ir/colbertv2.0` is used, encoding each company summary into a sequence of 128-dimensional per-token embeddings via the model’s linear projection layer. Due to indexing and library constraints on the cluster used for this thesis, the official PLAID-indexed ColBERT pipeline was not used; instead, MaxSim scoring is implemented from scratch as a brute-force computation over all 98 716 companies for every query. This is computationally more expensive than an indexed implementation would be, but is exact rather than approximate, and is tractable at this corpus size.

Table 4.12: ColBERT (brute-force MaxSim) Recall@1000 (relaxed top-1 000 protocol) alongside the strongest single-vector baselines.

Method	Precision@ <i>k</i>				Recall@1000
	10	50	100	1000	
MiniLM (Section 4.3.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.7)	0.946	0.936	0.925	0.768	0.768
ColBERT (MaxSim)	0.936	0.931	0.925	0.773	0.773

Finding. ColBERT’s Recall@1000 (0.773) is the highest of any single retrieval channel evaluated in this thesis, narrowly ahead of GTE-large (0.768) and clearly ahead of

MiniLM (0.741). Its NDCG@10 (0.937), however, is lower than both MiniLM (0.975) and GTE-large (0.946), meaning ColBERT retrieves a broader set of relevant companies overall but ranks the very top results somewhat less precisely. Token-level interaction therefore does deliver a genuine, if modest, recall improvement over the best single-vector models, though not one large enough on its own to close the gap to the hybrid rerank pipelines described in Section 4.19.

4.19 Hybrid Full-Pool Reranking

Section 4.10 showed that fusing BM25 and MiniLM with Reciprocal Rank Fusion, followed by reranking only the top-50 fused candidates, essentially reproduces MiniLM’s own quality (NDCG@10 = 0.975) without improving Recall@1000. Section 4.16 then showed that the *union* of several channels’ top-1 000 candidate pools reaches a much higher oracle recall ceiling than any single channel. Together, these two findings motivate a different hybrid design: rerank the *entire* union pool of several dense channels with a cross-encoder, rather than fusing rankings and reranking only a shallow, fixed-size slice of the result.

Implementation. For each query, the top-1 000 results from a set of dense channels are combined by simple set union (no rank fusion, no truncation), giving an unranked candidate pool that is typically 1 300–1 800 companies. A cross-encoder reranker, `BAAI/bge-reranker-v2-m3` (BAAI 2024), then scores every (query, candidate) pair in the full pool directly, and the top-1 000 by reranker score are kept as the final result. Four variants are compared, adding one channel at a time:

- **2-way:** MiniLM + Linq-Embed-Mistral.
- **3-way:** 2-way + GTE-large.
- **4-way:** 3-way + OpenAI-large.

Table 4.13: Full-pool reranking (relaxed top-1 000 protocol), compared against the shallow RRF + top-50-rerank baseline from Section 4.10. Bold indicates the best value.

Method	P@10	P@100	P@1000	R@1000	Pool
RRF + top-50 (§4.10)	0.980	0.940	0.782	0.782	50
2-way (MiniLM+Linq)	0.955	0.950	0.802	0.802	1 300
3-way (+GTE-large)	0.955	0.950	0.804	0.804	1 424
4-way (+OpenAI-large)	0.955	0.950	0.802	0.802	1 600

Finding. Reranking the full union pool instead of a fixed top-50 slice raises Recall@1000 substantially ($0.782 \rightarrow 0.802$, 2-way), at the cost of a small drop in NDCG@10 ($0.979 \rightarrow 0.955$): the reranker now has many more genuinely difficult candidates to rank, so its ordering of the very top results is slightly less precise, even though it recovers more of the true relevant set overall. Adding a third channel (GTE-large) improves Recall@1000 further, to 0.804, the best result obtained by any hybrid pipeline built from off-the-shelf pretrained components in this thesis. Adding a fourth channel (OpenAI-large) does *not* improve on this – Recall@1000 is unchanged at 0.802, within noise of the 3-way result.

Interpretation: channel saturation. This flat 3-way $\rightarrow$ 4-way result is an important turning point in this thesis’s experimental narrative. Section 4.16 showed that combining *more* candidate-generating channels reliably raises the oracle recall ceiling; here, adding a fourth channel to an already capable 3-way pool does not translate into a better final result. This suggests that candidate coverage is no longer the limiting factor for this pipeline: the union pool already contains the relevant companies needed to reach a substantially higher recall than 0.804 (recall Section 4.16’s six-channel oracle ceiling of 0.936) – and the reranker itself is now the bottleneck preventing the pipeline from closing that gap. This motivates two follow-up questions, addressed in the next two sections: is the specific reranker *model* the limiting factor (Section 4.20), or is a fundamentally different fusion approach needed (Section 4.21)?

4.20 Reranker Ablation: Isolating the Bottleneck

Given the channel-saturation finding in Section 4.19, the natural next question is whether the specific reranker model used throughout this chapter, BAAI/bge-reranker-v2-m3, is itself limiting recall, or whether reranker choice matters little once the candidate pool is fixed.

Implementation. BAAI/bge-reranker-v2-m3 (built on an XLM-RoBERTa backbone) is replaced with mixedbread-ai/mxbai-rerank-large-v1 (Mixedbread AI 2024) (built on a DeBERTa-v2 backbone, an architecturally distinct, comparably-sized cross-encoder), applied to the *exact same* 3-way union pool from Section 4.19. Only the reranker model changes; the candidate pool, corpus, and queries are identical.

Table 4.14: Reranker ablation Recall@1000: identical 3-way candidate pool, two different reranker models (relaxed top-1 000 protocol).

Reranker	P@10	P@100	P@1000	R@1000
BAAI/bge-reranker-v2-m3 (§4.19)	0.955	0.950	0.8042	0.8042
mxbai-rerank-large-v1	0.968	0.940	0.8035	0.8035

Finding. Recall@1000 is essentially unchanged between the two rerankers (0.8042 vs. 0.8035, a difference of 0.0007, well within query-level noise). NDCG@10 does improve modestly with the alternative reranker (0.955 $\rightarrow$ 0.971), indicating that `mxbai-rerank-large-v1` orders the very top results somewhat better, without changing which companies land in the top-1000 overall. The conclusion is that reranker *model choice* is not the bottleneck identified in Section 4.19: the remaining gap to the oracle ceiling is not a matter of picking a better off-the-shelf cross-encoder. This points towards a more fundamental limitation: either genuinely hard, vocabulary-mismatched cases that no cross-encoder scoring the `summary` field alone can resolve, or a fusion mechanism (simple set union plus independent reranking) that is itself insufficiently expressive, motivating the learned approach in Section 4.21.

4.21 Learned Fusion Ranker

Every hybrid pipeline so far combines *frozen*, independently pretrained components: candidate lists are pooled by simple union, and a pretrained cross-encoder, never fit to this task’s own queries or labels, reranks the result. This section instead treats fusion as a supervised learning-to-rank problem (Liu 2009): a model is trained directly on this thesis’s own 101 queries and their pseudo-relevance labels to learn how to combine multiple retrieval signals, rather than relying on a fixed, hand-designed combination rule.

Implementation. For every (query, candidate) pair in the same 3-way union pool used in Section 4.19 (with BM25 added as a fourth, lexical, signal), ten features are computed: the raw similarity score and inverse rank (1/rank) from each of MiniLM, Linq-Embed-Mistral, GTE-large, and BM25; the `BAAI/bge-reranker-v2-m3` score from Section 4.19 (included as one input feature among ten, not as the final decision rule); and a count of how many channels retrieved the candidate at all. Two models are trained on these features using `scikit-learn` (Pedregosa et al. 2011): a logistic regression, and a histogram-based gradient-boosted tree ensemble. Crucially, both are trained and evaluated using 5-fold cross-validation grouped by `query_id`: for each fold, the model is fit on candidates from 80% of queries and scores held-out candidates from the remaining 20%, so that no query is ever scored by a model that has seen its own label during training. Final metrics are computed by concatenating the held-out predictions from all five folds, giving a complete, leakage-free evaluation over all 101 queries.

Finding. The gradient-boosted learned fusion ranker reaches Recall@1000 = 0.823, the best result obtained anywhere in this thesis, improving on the best pretrained-component pipeline (0.804) by +0.019. For scale, this gain is roughly 27 times larger than the 0.0007 movement observed when merely swapping reranker models in Section 4.20; this is a substantive improvement attributable to training on the task’s own data, not noise. NDCG@10 also improves (0.955 $\rightarrow$ 0.979).

4 Experiments

Table 4.15: Learned fusion ranker Recall@1000 (held-out, cross-validated predictions; relaxed top-1 000 protocol), compared against the best pretrained-component pipeline from Section 4.19. Bold indicates the best value.

Method	P@10	P@100	P@1000	R@1000
3-way + BGE reranker (§4.19)	0.955	0.950	0.804	0.804
Learned fusion, logistic regression	0.968	0.953	0.821	0.821
Learned fusion, gradient-boosted trees	0.977	0.955	0.823	0.823

Interpretation: what the model learned. The trained logistic regression’s coefficients offer a direct, interpretable view into which signals matter most for this task: MiniLM’s *inverse rank* has by far the largest coefficient magnitude (+5.01), exceeding every raw similarity score *and* the pretrained reranker’s own score (+0.16). In other words, where a candidate is ranked by MiniLM is a more informative signal for final relevance than the pretrained cross-encoder’s score: a concrete, non-obvious finding that would not be visible from simply combining components by hand. This supports the conclusion drawn in Section 4.20: the pretrained reranker was not extracting the maximum available signal from the candidate pool, and a lightweight model trained on this task’s own data closes a meaningful part of the remaining gap to the oracle ceiling from Section 4.16.

Bibliography

- BAAI. bge-reranker-v2-m3. <https://huggingface.co/BAAI/bge-reranker-v2-m3>, 2024. Model card.
- Steven Bird, Ewan Klein, and Edward Loper. NLTK: the natural language toolkit, 2009.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. ACM, 2020. doi: 10.1145/3397271.3401075.
- Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331, 2009. doi: 10.1561/15000000016.
- Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- Mixedbread AI. mxbai-rerank-large-v1. <https://huggingface.co/mixedbread-ai/mxbai-rerank-large-v1>, 2024. Model card.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent

Bibliography

- Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Ellen M Voorhees. The trec-8 question answering track report. In *Proceedings of TREC*, 1999.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Simlm: Pre-training with representation bottleneck for dense passage retrieval, 2022.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 641–649. ACM, 2024. doi: 10.1145/3626772.3657878.
- Justin Zobel. *Writing for Computer Science*. Springer London, 2nd edition, 2004. doi: <https://doi.org/10.1007/978-0-85729-422-7>.
- Justin Zobel. *Writing for Computer Science*. Springer London, 3rd edition, 2014. doi: <https://doi.org/10.1007/978-1-4471-6639-9>.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?

Unterschrift

Mannheim, den July 24, 2026